EduLang

Educational Programming Language Compiler

TEAM - 6

Guide – Dr. V.Anu Ragavi

Team member’s :

1. Abhinav - 2420090086
2. Akshith - 2420030207
3. B.Vishvak sai - 2420090024

A custom language with a visual, step-by-step compiler pipeline

Project Overview

What is EduLang?

EduLang is a small, custom-designed programming language.

It supports variables, input/output, conditions, loops and arrays.

A compiler dashboard demonstrates how source code moves through each stage.

The project is designed to make compiler concepts easier to understand.

Problem Statement

Why build EduLang?

Compiler design contains several abstract stages that can be difficult for beginners to visualize.

Traditional explanations often show separate concepts without connecting them to one program.

Students need a simple way to observe tokens, syntax trees, semantic checks and generated code.

EduLang combines these concepts in one interactive learning environment.

EduLang Language Features

Custom syntax and supported constructs

Data types: integer, decimal, String, boolean and character

Input and output: Input(variable), Output(expression)

Control flow: if / else, for ... to ... and while

Arrays: declaration, initialization and indexed access

Comments: single-line comments using //

No main() function or user-defined functions in version 1.0

Sample EduLang Program

A short example using variables, conditions and output

```
integer age = 20
String name = "Abhi"

if age >= 18 {
  Output("Adult")
  } else {
  Output("Minor")
  }

Output(name)
```

Compiler Architecture

The path from source code to final output

1. Source Code
EduLang input

2. Lexical Analysis
Token generation

3. Syntax Analysis
Parsing and AST

4. Semantic Analysis
Types and declarations

5. Execution
Program interpretation

6. Intermediate Code
TAC generation

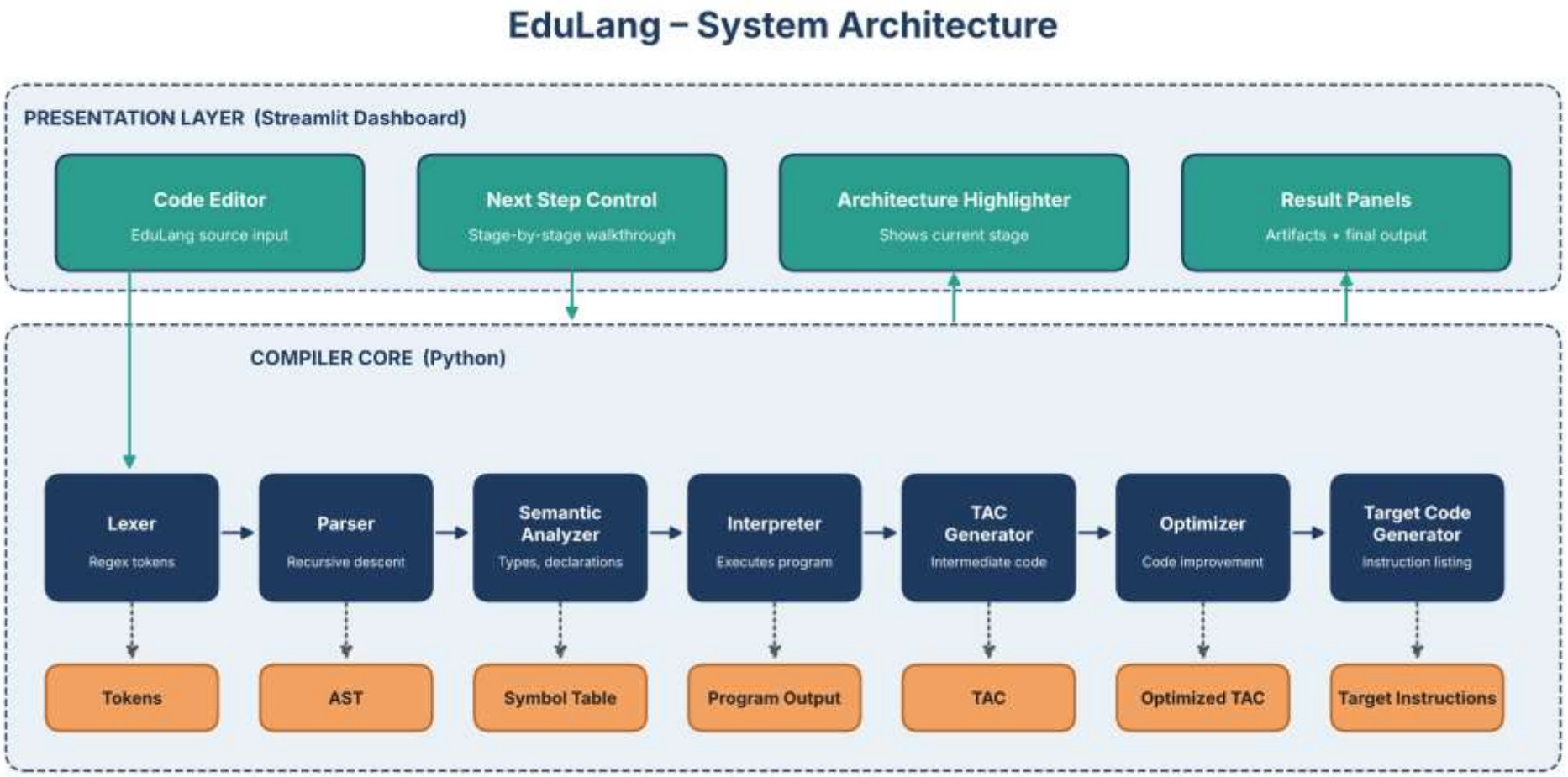
7. Optimization
Code improvement

8. Target Code
Instruction listing

9. Final Output
Program result

System Architecture

Presentation layer and compiler core of EduLang



Source code flows left-to-right through each compiler stage; every stage's artifact (orange) is returned to the dashboard for display.

Interactive Compiler Dashboard

Step-by-step walkthrough

The user enters a program in the EduLang code editor.

The architecture highlights the current compiler stage.

A Next Step button reveals the result of each stage.

The dashboard displays tokens, AST, symbol table, TAC, optimized listing and target instructions.

The final program output appears at the end of the walkthrough.

Technology Stack

Tools used to implement the prototype

Python – lexer, parser, semantic checks and interpreter logic

Streamlit – interactive web dashboard and step-by-step interface

Regular expressions – token recognition in lexical analysis

Recursive-descent parsing – syntax analysis and AST construction

Three Address Code style representation – intermediate code display

Conclusion & Future Scope

Summary and possible improvements

EduLang provides a practical demonstration of the major compiler stages.

Its custom syntax and dashboard make the compilation process easier to follow.

Future work: stronger optimization, more detailed semantic analysis and native target-code generation.

Additional features could include functions, nested scopes and richer error highlighting.